

AutoBench Service — Admin Guide

Last modified: 2026-09-30T15:37:05Z

Hand-maintained, unlike the generated `results/12run-*.md` files which stamp themselves. Bump the line above when you edit this guide.

How the AutoBench Service gets onto a cluster and stays trustworthy there: prerequisites, the environment that differs between platforms, the Helm chart, and the verification that actually proves the install. *Driving* the Service once it answers is a different document — [DEVELOPER_GUIDE.md](#).

- **Every host below is a placeholder on `example.com`** — this repo is public, so it names platforms rather than endpoints. Route *shapes* are real (`<service>-<namespace>.apps.<cluster>.example.com`), so substitute your own domain and the recipes work unchanged.
- **The cluster names are samples, and so is every endpoint they stand for.** `ykt2` / `ykt3` / `ykt5` are the OpenShift clusters this project happens to run on, reached from the Internet; the KinD cluster is a laptop on an organisation intranet. Nothing here requires any of them, or a particular LLM service (§3.5) — they are the worked examples, and the chart is deliberately generic.
- **No secret value appears in this guide, and none should appear in your terminal.** Every script here reads credentials from the environment or from a `chmod 600` file and reports them only as a truncated SHA-256. If you find yourself about to `echo` one, don't.
- Two install paths exist and both are supported: the **Helm chart** at `deploy/helm/autobench` (§5) and the **raw manifests** under `deploy/` (§8). They render the same objects, and that equivalence is enforced by a check, not asserted.

In a hurry? §2.4 is the one command that tells you whether the cluster is ready, §5 is the install, §6 is the only verification that means anything.

Contents

- [1. What you install, and what you don't](#)
 - [The two cluster shapes](#)
- [2. Prerequisites](#)
 - [2.1 Rossoctl v0.8.0 or later](#)
 - [2.2 Workstation tooling](#)
 - [2.3 Cluster-side checklist](#)
 - [2.4 The preflight script](#)
- [3. Environment variables](#)
 - [3.1 Install-time — your shell only, never a values file](#)
 - [3.2 The Service pod](#)
 - [3.3 The workload pods — injected, not configured](#)
 - [3.4 KinD and OpenShift differ — and the differences fail silently](#)
 - [3.5 The LLM gateway: two named profiles](#)
 - [3.6 The `benchmarker` password: how it gets in, and how it is checked](#)
 - [3.7 MLflow: already there, or installed by us — and how you find out which](#)

- [4. The instance-config Secret](#)
- [5. Installing with Helm](#)
 - [5.1 OpenShift](#)
 - [5.2 KinD](#)
 - [5.3 Adopting an install made from the raw manifests](#)
 - [5.4 The IBAC judge — installed and uninstalled with the release](#)
 - [5.5 Upgrade, rollback, uninstall](#)
- [6. Verifying the install](#)
- [7. Symptoms that lie](#)
- [8. Appendix: the raw-manifest path, and two KinD-only objects](#)

1. What you install, and what you don't

The Service is a single stateless pod. It holds no benchmark data and no cluster credential: it authenticates to Rossoctl per request with its own ROPC identity, and Rossoctl performs every cluster operation server-side. Nothing here shells out to `kubect`l or `oc`.

object	name	created by	notes
Deployment	autobench-service	chart / deploy/deployment.yaml	1 replica, read-only root filesystem, arbitrary-UID safe
Service	autobench-service	chart / deploy/service.yaml	ClusterIP :8080, port named http
Route	autobench	chart (platform: openshift)	edge TLS; host generated as autobench-<namespace>.apps.<cluster>
HTTPRoute	autobench	chart (platform: kind)	attaches to the shared istio gateway
Secret	autobench-instances	you, out-of-band (§4)	ROPC credentials — never passes through Helm values
RoleBinding	autobench-service-mlflow-trace-writer	chart (mlflowTraceWriter.enabled)	one per MLflow workspace namespace; lets the Service write its own spans. On by default in values-openshift.yaml — see the warning below
Deployment + Service	ibac-judge	chart (ibacJudge.enabled, off by default)	needed only by the plugin legs #5–#8 (§5.4)

The judge is in that table, rather than in the platform list below, on purpose: **Rossoctl ships none**. `rossoctl-platform-config` arrives with every `ibac.*` field empty, and a cluster with no benchmark workloads never needs one — so the judge exists for benchmarking, AutoBench installs it, and `helm uninstall` takes it away again (§5.4).

Everything else the Service depends on belongs to the platform and must exist **before** the install is useful:

- a Keycloak user (`benchmarker` by convention) with the `rossoctl-operator` realm role;

- `openai-secret` (`apikey`) and `hf-secret` (`hf-token`) in **every namespace you deploy benchmarks into** — `team1` unless you have changed the specs, since all 24 entries in `reference/run12_specs.json` name it;
- an OTEL collector that writes to an MLflow the Service can read (§3.4).

The two cluster shapes

shape	example	consequence
single-cluster — Service and agents together	KinD, ykt5	agents reach the collector over service DNS; no extra ingress
split — Service here, agents there	ykt3 hosts the Service, agents run on ykt2	the instance config carries <code>agent_endpoint_template/mcp_endpoint_template</code> , and the agents export telemetry back through an edge Route on <code>:443</code> whose target port resolves to the collector's 8335

The shape is not a chart value. It lives entirely in the instance config, so one chart installs both.

2. Prerequisites

2.1 Rossoctl v0.8.0 or later

This is the hard one, because falling short of it does not error. An older operator accepts the deploy request the Service sends and **silently drops fields it does not know** — `k8sResourceLimits` among them — so the benchmark comes up in a shape nobody configured and every subsequent measurement is quietly off.

Read the version off the **backend**. The operator subchart carries its own, lower, version line, and comparing that one rejects a perfectly current cluster:

```
oc -n rossoctl-system get deploy rossoctl-backend \
-o jsonpath='{.metadata.labels.app\.kubernetes\.io/version}' # 0.8.0-rc.2
```

A release-candidate suffix counts: `0.8.0-rc.2` satisfies "`≥ 0.8.0`". Both `reference/preflight.py` and `reference/ocp-service-bootstrap.sh` assert this before doing anything else.

2.2 Workstation tooling

tool	needed for	note
<code>kubectl</code> (or <code>oc</code>)	everything	<code>oc</code> only for Route conveniences; <code>kubectl</code> can do it all
<code>helm</code> $\geq$ 3.8	§5	3.8 is the floor for the <code>OCI/--kube-context</code> behaviour used here
<code>jq</code> , <code>curl</code>	the bootstrap scripts	secrets move through <code>jq</code> via the <i>environment</i> , never <i>argv</i>
<code>python3</code>	<code>preflight.py</code> , <code>helm-parity-check.py</code> , <code>gen_toc.py</code>	<code>kind-collector-mlflow.py</code> also needs <code>pyyaml</code>

tool	needed for	note
kind + a container engine	the KinD path only	the image is built locally and kind loaded

2.3 Cluster-side checklist

Each row is a failure that has actually been shipped, and the right-hand column is why you cannot find it by reading logs at the layer where it hurts.

prerequisite	where	if missing
Rossoctl $\geq$ v0.8.0	rossoctl-system	fields dropped from the deploy request, silently (§2.1)
agentruntimes + agentcards CRDs	cluster	/deploy fails at the operator
benchmarker user with a password credential, ROPC enabled on the client	realm	token request 400s; on this realm a missing firstName/lastName yields Account is not fully set up. Checked in two tiers, because ROPC alone cannot distinguish a wrong password from an absent one (§3.6)
rossoctl-operator realm role on that user	realm	/deploy surfaces the operator's 403 as a 502
openai-secret / apikey	team1	empty or foreign key $\Rightarrow$ a 401 per completion, mid-run ; the leg finishes with zeroes rather than failing
hf-secret / hf-token (may be empty)	team1	MCP pod CreateContainerConfigError, the agent then crash-loops against it, and the <i>run</i> reports 424
OTEL collector, HTTP receiver on :8335	rossoctl-system	an agent pointed at 4318 hard-fails into CrashLoopBackOff
an MLflow the collector writes to and the Service can read	varies (§3.4)	the run passes with model: "unknown" and every token count 0

On the collector port: 8335 and only 8335. Every cluster's `otel-collector-config` declares receivers on 4317/4318, and reading that ConfigMap will tell you to use 4318. It is wrong — the Deployment moves the HTTP receiver with a command-line override, so 4318 never listens:

```
oc -n rossoctl-system get deploy otel-collector \
-o jsonpath='{.spec.template.spec.containers[0].command}'
# ["/otelcol-contrib","--config=/etc/otelcol-config/base.yaml",
#  "--set","receivers::otlp::protocols::http::endpoint=0.0.0.0:8335"]
```

The same principle applies to MLflow: read the target and the experiment id from the collector's own exporter block rather than choosing them. `traces_endpoint` and the `x-mlflow-experiment-id` header are what the *writer* uses, and an instance config that disagrees points the Service at an experiment nothing writes to.

2.4 The preflight script

`reference/preflight.py` checks every row of §2.3 against the live cluster. It makes **no cluster writes and no deploys**, and prints every credential as an 8-char SHA-256 prefix unconditionally, with no attempt to decide which values are sensitive. It is read-only with exactly one exception, the final MLflow section: that asks the deployed Service for `GET /mlflow/health`, which writes one synthetic trace to MLflow (§3.7). --

`skip-mlflow-probe` turns it off, at the price of the install no longer being gated on the failure that is hardest to notice.

```
python3 reference/preflight.py --platform kind --context kind-rossoctl
python3 reference/preflight.py --platform openshift --context <ykt5-ctx>
python3 reference/preflight.py --platform openshift --context <ykt3-ctx> \
    --workload-context <ykt2-ctx> # split shape
python3 reference/preflight.py --json # machine-readable
```

Exit status is 0 when nothing FAILED; warnings do not fail the run. Thirteen sections, in the order a request travels: tooling, cluster reachability, Rossoctl version and CRDs, namespaces, workload secrets, the collector, MLflow, ingress, the instance config, identity, the MLflow round trip, an audit of any existing install, and the local chart.

```
Workload secrets (per team namespace)
ok    team1/openai-secret apikey - sha8 0239f193
ok    team1/hf-secret hf-token - empty value (fine)

OTEL collector (the write half of the telemetry chain)
ok    collector HTTP receiver on :8335 - read from the Deployment's command
ok    collector traces_endpoint - http://mlflow-reader.rossoctl-
    system.svc.cluster.local:5000/v1/traces
ok    collector x-mlflow-experiment-id - 0

MLflow (the read half - this is what turns a run into a token report)
ok    MLflow traces API answers 200 - experiment 0, 1 trace(s) visible
ok    collector exports to mlflow-reader
...
MLflow round trip (the only check that reaches the write half end to end)
ok    MLflow tracking URL (as the Service resolves it) - http://mlflow-reader.rossoctl-
    system.svc.cluster.local:5000
ok    MLflow credential mode - bearer_token
ok    MLflow auth - bearer obtained via bearer_token
ok    MLflow read - traces API answered; experiment 0 is non-empty
ok    MLflow write - probe spans exported
ok    MLflow round trip - probe trace readable after 2 attempt(s)
...
47 ok, 0 warning(s), 0 failure(s) - ready to install
A clean preflight is necessary, not sufficient: only a 1-task leg with a non-zero token row
proves the whole chain.
```

Three things about how it reads the cluster are worth knowing, because each was a false alarm first:

- **Ports and targets come from live objects**, never from a manifest or a ConfigMap — the collector's port from the Deployment's `command`, MLflow's identity from the exporter.
- **Route has no CRD**. It is served by the aggregated openshift-apiserver, so `get crd routes.route.openshift.io` reports it missing on a cluster that plainly serves Routes. The script asks `api-resources --api-group=route.openshift.io` instead.
- **A split shape is not a broken one**. With `agent_endpoint_template` in the instance config and no `--workload-context`, the workload checks are *skipped* rather than run against the wrong cluster — and a `workload_otel` endpoint on `:443` is resolved through the Route to the collector's port instead of being rejected.

The **identity** section is the one that needs a credential from you, and it is not optional in practice: every `/deploy` the Service makes is a ROPC login as `benchmarker`, so a password that is absent, stale or blocked by a required action does not degrade the install — it makes every run fail with a 502 wrapping a 403. Pass it in and preflight logs in for real:

```
python3 reference/preflight.py --platform kind --context kind-rossoctl \
  --password-file ~/.rossoctl-kind/benchmarker.pass
```

With no password at all the script falls back to the one inside the instance Secret, which is the copy that actually matters. §3.6 covers the four intake routes, the two tiers of the check, and what each failure cause means.

3. Environment variables

There are three populations, and conflating them is how a key ends up in the wrong cluster. They are, in order: what you export **to run the installer**, what the **Service pod** reads, and what the operator injects into the **workload pods**.

3.1 Install-time — your shell only, never a values file

Nothing in this table is a Helm value, a ConfigMap, or a committed file. Each is read by a script, used once, and hashed if it is reported at all.

variable	used by	notes
KC_SERVICE_USERNAME	both bootstrap scripts, <code>preflight.py</code>	the ROPC login the Service uses against Rossoctl (default <code>benchmarker</code>); <code>--username</code> overrides
KC_SERVICE_PASSWORD	both bootstrap scripts, <code>preflight.py</code>	required ; written into the instance file, never echoed. <code>--password-file</code> / <code>--password-stdin</code> / <code>--password</code> take precedence (§3.6)
KC_SERVICE_CLIENT_SECRET	both bootstrap scripts	only if the Keycloak client is confidential
KC_USER_PASSWORD	<code>kind-post-setup.sh</code>	the password to <i>seed</i> ; the same three flags override it, and it falls back to <code>~/.rossoctl-kind/benchmarker.pass</code> (<code>KC_CRED_FILE</code>)
KC_ADMIN_PASSWORD	<code>kind-post-setup.sh</code> , <code>preflight.py</code>	optional — read from the in-cluster <code>keycloak-initial-admin</code> Secret when unset. In <code>preflight.py</code> it enables the Admin-API tier of §3.6
KC_ADMIN_USER	<code>preflight.py</code>	optional, default <code>admin</code> — the master-realm <code>admin</code> the Admin-API tier logs in as
KC_ISS	<code>preflight.py</code>	optional — the issuer to log in against, when neither <code>--iss</code> nor the instance file supplies one
LLM_PROFILE	both bootstrap scripts, <code>kind-post-setup.sh</code> , <code>preflight.py</code>	<code>intranet</code> <code>internet</code> — selects one of the two gateway variable sets (§3.5)
INTRANET_LLM_* / INTERNET_LLM_*	<code>reference/llm-profiles.sh</code> , read by all of the above	the profiles themselves: <code>base</code> , <code>model</code> , <code>key file</code> , optional <code>bypass list</code> (§3.5)

variable	used by	notes
BM_WORKLOAD_LLM_KEY	kind-post-setup.sh	the workload LLM key; falls back to the selected profile's key file, else ~/.rossoctl-kind/litellm.key (LLM_KEY_FILE)
S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY	ocp-service-bootstrap.sh	override --s3-from, which otherwise copies them in-process from an existing instance file
WORKLOAD_LLM_API_BASE, WORKLOAD_LLM_MODEL	both bootstrap scripts	set by LLM_PROFILE when you use one; an explicit export wins. The gateway has no default on purpose: this repo is public
MLFLOW_URL, MLFLOW_EXPERIMENT_ID, MLFLOW_WORKSPACE, MLFLOW_TOKEN_SECRET	ocp-service-bootstrap.sh	the id and workspace default to what the collector exports
WORKLOAD_OTEL_ENDPOINT, WORKLOAD_OTEL_INSECURE, WORKLOAD_AGENT_RUNNER	both bootstrap scripts	see §3.4 on the runner
IMAGE, CLUSTER, KUBE_CONTEXT, REALM, CLIENT, KC_HOST	kind-post-setup.sh	plain overrides, no secrets

Two conventions here are deliberate and both cost real time when broken:

The LLM key is `BM_WORKLOAD_LLM_KEY`, not `OPENAI_API_KEY`. That name is commonly exported in a developer's shell profile for an unrelated provider, and the script writes whatever it finds into cluster Secrets. A namespaced name cannot be inherited by accident. (Inside the pod the value still arrives as `OPENAI_API_KEY` — the registry maps it from the Secret's `apikey`.)

Credential files are refused unless they are `600` or `400`, and they are meant to outlive the cluster: neither the `benchmarker` password nor the LLM key changes across a rebuild.

```
umask 077; mkdir -p ~/.rossoctl-kind
printf '%s' '<benchmarker password>' > ~/.rossoctl-kind/benchmarker.pass
printf '%s' '<llm key>' > ~/.rossoctl-kind/litellm.key
```

3.2 The Service pod

The container reads exactly two environment variables from the Deployment, and both are set by the chart:

variable	value	why
SERVICE_INSTANCES_DIR	/etc/service/instances	where the autobench-instances Secret is mounted read-only
SERVICE_PORT	8080	read by the entrypoint (uvicorn), not by the settings model

Everything else is a `SERVICE_`-prefixed setting with a working default, overridable through `extraEnv` in the values file. These are the ones worth knowing:

setting	default	when you would change it
SERVICE_LOG_LEVEL	INFO	DEBUG while diagnosing an auth or deploy failure

setting	default	when you would change it
SERVICE_HTTP_TIMEOUT_SECONDS	30	a slow Rossoct! API
SERVICE_JWKS_CACHE_SECONDS	300	rarely
SERVICE_EXPORT_SETTLE_MAX_SECONDS	20	a fast run can be exported <i>before</i> its child spans land, yielding model: "unknown" and zero tokens; the Service re-reads until the record set stops growing, and this is that budget
SERVICE_EXPORT_SETTLE_INTERVAL_SECONDS	2	with the above
SERVICE_SPAN_REPORT_MAX_ROWS	200000	only to cap a pathological trace — the export is built in memory

```
# values fragment
extraEnv:
  - name: SERVICE_LOG_LEVEL
    value: DEBUG
```

The settings model uses Pydantic's `extra="ignore"`, so a **misspelled `SERVICE_*` name is accepted and discarded without a word**. If a setting seems not to take effect, suspect the spelling before the code.

3.3 The workload pods — injected, not configured

You do not set these. The Service derives them per deploy from the benchmark definition (`src/autobench/benchmarks/registry.py`) and the instance config, which is why an agent's environment cannot drift from the run that measured it. They are listed so you can recognise them in a pod spec:

variable	source	note
OPENAI_API_KEY	secretKeyRef → openai-secret / apikey	never a literal value in the pod spec
HF_TOKEN	secretKeyRef → hf-secret / hf-token	gsm8k only; presence is the requirement, not content
OPENAI_API_BASE, LLM_API_BASE	instance workload_llm.api_base	any inherited values are dropped and re-injected
EXGENTIC_DEFAULT_RUNNER	instance workload_agent_runner	§3.4
EXGENTIC_OTEL_ENABLED	true when workload_otel.enabled	
OTEL_EXPORTER_OTLP_ENDPOINT / _PROTOCOL / _INSECURE	instance workload_otel	the endpoint is the :8335 one

3.4 KinD and OpenShift differ — and the differences fail silently

The Service's own objects are nearly identical across platforms. What differs is everything it points *at*.

	KinD	OpenShift	if you get it wrong
LLM gateway	not a platform difference — either platform may use a service on the intranet or on the Internet, subject to the asymmetric rule in §3.5	ditto	no two services share a key table, so a key moved across does not fail closed — it 401s per completion, mid-run
openai-secret	a key issued by that cluster's own service	ditto	same
MLflow read path	mlflow-reader (§8), no auth	the cluster's own MLflow on :8443 with a ServiceAccount bearer and insecure_tls	a refused read is invisible: the run passes, every token count is 0
MLflow experiment	0	1, workspace team1 on ykt5	identical signature to the above
Keycloak dial	the backchannel service DNS — iss is unreachable in-cluster, since *.localtest.me resolves to pod loopback and there is no CoreDNS rewrite	the iss Route itself	JWKS and ROPC both fail at startup
collector endpoint	http://otel-collector.rossoc1-system.svc.cluster.local:8335	the same on a single-cluster install; an edge Route on :443 when the agents live elsewhere	unreachable collector ⇒ agent CrashLoopBackOff, surfacing as a 424 on the run
pod security context	UID/GID/fsGroup pinned to 10001/0/10001	runAsNonRoot + seccomp only	pinning 10001 can be rejected when it falls outside the project's allocated UID range
ingress	HTTPRoute on the shared gateway	Route, edge TLS	—

On OpenShift the Service's own span export needs two more things, and each fails silently. Reaching MLflow is TLS; being allowed to write to it is RBAC. The Service reads MLflow to build `report.ndjson`, but it also *emits* four spans of its own — `Agent.Session` and its three children — straight to the OTLP endpoint. If either half is missing, the export fails, MLflow never receives `Agent.Session`, and a trace without that root span is **dropped** when the report is assembled: the run succeeds, reports `pass_rate 1.0`, and publishes a **zero-byte** `report.ndjson` and `token_report.ndjson`. Nothing errors. The measurement is simply not there.

half	what it needs	rendered by
TLS — an in-cluster MLflow serves a cert signed by the OpenShift service CA, absent from the default trust store	OTEL_EXPORTER_OTLP_TRACES_CERTIFICATE=/var/run/secrets/kubernetes.io/serviceaccount/service-ca.crt	extraEnv in values-openshift.yaml; deploy/openshift/deployment-patch.yaml for the raw path
RBAC — writes are authorized against the MLflow operator's ClusterRole in the MLflow workspace namespace	a RoleBinding per workspace, granting mlflow-reader mlflow-operator-mlflow-integration	mlflowTraceWriter.enabled: true

It must be the OTLP-specific variable. `REQUESTS_CA_BUNDLE` looks like the obvious choice and is a trap: botocore reads it too, so it redirects the S3 client's trust store at the same time and every artifact upload then fails to validate AWS's public cert — turning an empty report into no artifacts at all. `preflight.py` checks

both halves, and matches the RoleBinding on the **grant** rather than its name, because a cluster set up before the chart existed has an equivalent binding under a different one. `mlflowTraceWriter.namespaces` lists MLflow **workspaces** on the Service's own cluster, not the namespaces agents run in — the same string on a single-cluster install, but in the split shape the agents are elsewhere while the MLflow being written to is here. A workspace missing from the list reproduces the empty report for that workspace only.

workload_agent_runner deserves its own warning. The correct value tracks an agent image pinned to `:latest`, and it has flipped between `direct` and `service` across rebuilds of that tag. With the current image, `service` is what emits agent spans; `direct` yields a clean run with **zero-token rows**, which is exactly what a broken MLflow looks like. Do not infer it — detect it, by looking for a non-zero token row in `report.ndjson` (§6).

3.5 The LLM gateway: two named profiles

The gateway is the one piece of configuration that is neither discoverable from the cluster nor shared between clusters, so it gets its own section — and it is configured as a **named profile** rather than as a set of per-cluster values, because the failure it causes is silent.

Any OpenAI-compatible **LLM or LiteLLM service** can serve a deployment; nothing in AutoBench requires a particular one, and the endpoints this project happens to use are examples, not part of the product. Your organisation's will be different services with different hosts, model catalogues and keys.

What decides which one a deployment may be pointed at is **the network the service sits on relative to the cluster**, not which Kubernetes the cluster runs. Both platforms occur in both places: an OpenShift cluster is often deployed on the organisation's intranet, and a KinD cluster runs on an Internet server as readily as on a laptop on the VPN. `platform` and `llmProfile` are therefore independent, and nothing derives one from the other.

The rule is **asymmetric**:

- An **Internet** cluster — KinD or OpenShift — must be configured with the access credentials for an LLM or LiteLLM service **on the Internet**. An intranet service is not routable from it.
- An **intranet** cluster — KinD or OpenShift — can be configured with the access credentials for an LLM or LiteLLM service **on the intranet or on the Internet**, whichever it is permitted to reach. So an intranet cluster has a genuine choice, and the profile records which way it went.

	intranet	internet
what it points at	an LLM or LiteLLM service on the organisation's intranet	one on the Internet
which clusters may declare it	intranet clusters only — KinD or OpenShift	any cluster, on either network
why not the other	an Internet cluster cannot route to an intranet service — connect times out	an intranet cluster <i>may</i> use this; it is only barred where egress to the Internet is
model catalogue	ids from <i>that</i> service	not the same ids
key table	its own	its own — this is the whole problem

The two variable sets

Each profile is one set of environment variables, distinguished only by prefix, defined in `reference/llm-profiles.sh`:

intranet	internet	what it is
INTRANET_LLM_API_BASE	INTERNET_LLM_API_BASE	the gateway origin — scheme + host, no path
INTRANET_LLM_MODEL	INTERNET_LLM_MODEL	a model id from that gateway's catalogue
INTRANET_LLM_KEY_FILE	INTERNET_LLM_KEY_FILE	chmod 600 file holding that gateway's key (default <code>~/.rossoctl-llm/<profile>.key</code>)
INTRANET_LLM_NO_PROXY	INTERNET_LLM_NO_PROXY	optional explicit egress-proxy bypass list; otherwise built for you

`LLM_PROFILE=intranet|internet` (or `--llm-profile`) selects one, and the scripts copy it into the canonical `WORKLOAD_LLM_API_BASE` / `WORKLOAD_LLM_MODEL` / `LLM_KEY_FILE` they already read. An explicitly exported `WORKLOAD_LLM_*` still wins, so nothing that worked before behaves differently.

Bases and model ids are not secrets, so keep them in a file instead of re-exporting per shell. The **keys stay out of it**, in their own per-profile files:

```
mkdir -p ~/.rossoctl-llm
cat > ~/.rossoctl-llm/profiles.env <<'EOF'
INTRANET_LLM_API_BASE=https://<intranet LLM service host>
INTRANET_LLM_MODEL=openai/aws/claude-haiku-4-5
INTERNET_LLM_API_BASE=https://<Internet LLM service host>
INTERNET_LLM_MODEL=openai/Azure/gpt-5-mini-2025-08-07
EOF
umask 077
printf '%s' '<intranet service key>' > ~/.rossoctl-llm/intranet.key
printf '%s' '<Internet service key>' > ~/.rossoctl-llm/internet.key
```

Per-profile key files are the point: no two services share a key table, so one file holding "the" key is exactly how a key gets used against the service that never issued it.

Selecting a profile for a deployment

The profile is declared **once**, as `llmProfile` in the chart values file, and everything else reads it from there:

```
# deploy/helm/values-kind.yaml           # deploy/helm/values-openshift.yaml
platform: kind                           # platform: openshift
llmProfile: intranet                     # llmProfile: internet
```

```
reference/ocp-service-bootstrap.sh --llm-profile internet --cluster ykt5 --context <ctx> ...
LLM_PROFILE=intranet reference/kind-post-setup.sh
python3 reference/preflight.py --values deploy/helm/values-kind.yaml --context kind-rossoctl
```

The chart **does not** render the gateway into the Service pod, and cannot: the Service reads the gateway from `workload_llm.api_base` in the instance-config Secret (§4) — not from its own environment — and injects `OPENAI_API_BASE` / `LLM_API_BASE` onto each workload pod at deploy time (§3.3). So `llmProfile` is a **declaration**, and it does two things that matter:

- the bootstrap scripts take the profile from it, so the instance file they write cannot disagree with the release installed beside it;
- `preflight.py` **FAILS** when the live instance file's `api_base` is not that profile's base:

```
FAIL keycloak.localtest.me_8080.json: LLM gateway matches the internet profile – instance
points
  at <internal host>, the internet profile declares <external host> – separate key tables,
so
  this 401s per completion mid-run rather than failing at deploy
```

That is the entire point of naming the profiles: it converts a mid-run 401 storm, whose only visible symptom is a leg that finishes with zeroes, into a pre-install error. `helm` rejects a mistyped profile for the same reason — `llmProfile: intranett` fails the render rather than being quietly ignored downstream.

Four details that have cost time

- **The key never travels in the instance file.** It lives only in `openai-secret` / `apikey` in each workload namespace, and reaches the pod as `OPENAI_API_KEY` through a `secretKeyRef`.
- **The proxy-bypass list is part of the gateway's configuration, not a separate concern.** Where the cluster injects an egress proxy, an in-network gateway base must be excluded from it or every completion leaves through a proxy that cannot reach it. `workload_llm.no_proxy` (and `disable_proxy: true`, which injects an empty `HTTP_PROXY` / `http_proxy`) are rendered onto the **agent** pod only. Both bootstrap scripts now build the list — loopback, the gateway host, the collector, and Keycloak's in-cluster service — where before only the OpenShift one did, so a KinD bootstrap without `--copy-from` silently lost it.
- **WORKLOAD_LLM_BASE was never a second setting.** It is an accepted alias for `WORKLOAD_LLM_API_BASE` in the KinD script, kept only so existing invocations keep working; the canonical name matches the instance-file field (`workload_llm.api_base`) on both platforms.
- **Do not verify a gateway with curl.** LiteLLM strips the `openai/` prefix, so the configured model name 403s by hand, and reasoning models reject `max_tokens` with a 400 (`max_completion_tokens` is the accepted field). The only trustworthy probe is a 1-task `gsm8k` leg (§6).

3.6 The `benchmarker` password: how it gets in, and how it is checked

This credential is load-bearing **per run**, not per install. The Service holds no cluster credential of its own: each `POST /benchmarks/{name}/deploy` is a fresh ROPC password grant as `benchmarker` against the realm, and the resulting token is what `Rossoctl` authorises. An install can therefore be completely healthy — pod Running, `/healthz` 200, `GET /benchmarks` listing every benchmark — and still fail every single run with a 502 that wraps Keycloak's 403.

There is no Keycloak object scoped to `team1`, incidentally: `team1` is a Kubernetes namespace, and the authorisation artifact for deploying into it is the **rossoctl-operator realm role** on the user in the `rossoctl` realm. So "is the password set for team1" is really five questions — does the user exist, is it enabled, does it have a `password` credential, is its profile complete enough for the realm to issue a token, and does it carry that role — and the checks below answer them separately.

Four ways in, in precedence order

Every script that needs the password takes the same four, most explicit first:

route	used as	notes
<code>--password-file F</code>	<code>preflight.py</code> , both bootstrap scripts, <code>kind-post-setup.sh</code>	preferred. The value never reaches <code>argv</code> or an environment. The file must be 600 or 400 or it is refused
<code>--password-stdin</code>	same	for a pipeline or a secret manager: <code>pass show ... script --password-stdin</code>
<code>--password P</code>	same	accepted, and warns every time: <code>argv</code> is world-readable through <code>ps</code> and <code>/proc/<pid>/cmdline</code> , and it lands in your shell history
the environment	<code>KC_SERVICE_PASSWORD</code> (bootstrap, <code>preflight</code>), <code>KC_USER_PASSWORD</code> (<code>kind-post-setup.sh</code>)	the original route, unchanged

Two conveniences on top: `kind-post-setup.sh` falls back to `~/.rossoctl-kind/benchmarkers.pass` (`KC_CRED_FILE`), so on KinD no credential flag is ever needed; and `preflight.py` falls back to the `service_credential` inside the live `autobench-instances` Secret, so it can check an install whose password you do not have to hand.

Only the **source** is printed, alongside an 8-char SHA-256 prefix of the value:

```
Identity - the benchmarker credential every /deploy authenticates with
ok    benchmarker password from file ~/.rossoctl-kind/benchmarkers.pass - user benchmarker,
      sha8 8bb6f116
```

That hash is also how the supplied password is compared against the Secret's copy without either being shown. **A mismatch is a FAILURE, not a warning** — the Secret's copy is the one the Service presents, so a green preflight next to a Service that 502s on every deploy would be worse than no check at all.

Because a trailing newline is welded onto the value by every editor, the file readers take the **first line only**. A password with `\n` on the end fails ROPC with exactly the message a wrong password gives.

Two tiers, because ROPC cannot tell three failures apart

Keycloak answers `invalid_grant` / "Invalid user credentials" identically for a **wrong password**, a user with **no password credential**, and **no such user**. Those need three different fixes, so `preflight.py` asks the Admin REST API first when it can, and the ROPC login second:

```
ok    user benchmarker exists in realm rossoctl - id e7557a3c-0a9f-4427-9d1a-3ab69727abc4
ok    password credential set on benchmarker - credential type(s): password
ok    ROPC login as benchmarker - against http://keycloak.localtest.me:8080/realms/rossoctl
ok    realm role rossoctl-operator
```

The admin tier needs `KC_ADMIN_PASSWORD`, or an in-cluster `keycloak-initial-admin` Secret (which is how KinD ships — nothing to supply there). Without one it is skipped with a note and the ROPC login stands alone as the behavioural check. When the tier *has* run, its finding is threaded into the failure text, which is the difference between a guess and an answer:

```
FAIL ROPC login as benchmarker - 'Invalid user credentials' - a password credential IS set on
    this
    user (checked above), so the password given to this script is not the one Keycloak holds.
```

The causes and their fixes:

cause	what it means	fix
Invalid user credentials, password credential is set	the password you supplied is not the one Keycloak holds	supply the right one, or re-seed with <code>reference/keycloak-ensure-user.sh</code>
Invalid user credentials, no password credential	the user exists with no password at all	<code>reference/keycloak-ensure-user.sh</code>
Invalid user credentials, no such user	not in this realm	<code>reference/keycloak-ensure-user.sh</code> ; check you have the right realm
Account is not fully set up	the password may be correct — a required action is pending, or <code>firstName/lastName</code> are unset, which this realm's user profile demands before it issues a token	set both (<code>kind-post-setup.sh</code> patches them), clear the required action
<code>unauthorized_client / invalid_client</code>	the client is confidential, or has <code>directAccessGrantsEnabled: false</code>	<code>KC_SERVICE_CLIENT_SECRET</code> , or <code>keycloak-ensure-user.sh</code> , which enables Direct Access Grants idempotently
login ok, role missing	every <code>/deploy</code> will 403	grant the <code>rossoctl-operator</code> realm role

Where in the flow each script checks

- **kind-post-setup.sh** seeds the user, then patches `firstName / lastName`, then grants the role. It deliberately calls `keycloak-ensure-user.sh` *without* `--verify`: before that patch the realm refuses a token no matter how correct the password is, so verifying there would fail on a healthy setup.
- **kind-service-bootstrap.sh** verifies at the end of that chain, immediately before it writes the password into the instance file, and **dies** rather than writing an unusable one. It also warns when the token carries no `rossoctl-operator` role. `SKIP_CRED_CHECK=1` writes the file anyway — only useful when Keycloak is not reachable from where you are running.
- **ocp-service-bootstrap.sh** reports the source as a precheck row and logs in as another, alongside the Rossoctl-version and gateway-profile checks.
- **preflight.py** is the read-only pre-`helm` gate, and the only one with the admin tier.

3.7 MLflow: already there, or installed by us — and how you find out which

MLflow is the one dependency whose *existence* varies by cluster rather than by configuration. On OpenShift it is normally **pre-installed** — `rossoctl-deps` or RHOAI puts it there, SAR-gated on `:8443`, owned by whoever installed it. On KinD it normally has to be **installed by us**, because the MLflow `rossoctl-deps` ships runs `mlflow-oidc-auth` and rejects both the collector's export and the Service's read (§8). Neither is a rule: a KinD cluster on an Internet server may well be pointed at an MLflow that already exists, and then installing a second one is wrong.

So the install path **checks** instead of assuming:

script	what it does now
kind-post-setup.sh – –install-mlflow auto always never	auto (the default) applies deploy/kind/mlflow-reader.yaml only when nothing is already serving MLFLOW_URL; it logs which way it went and why. never is for a cluster whose MLflow is external — the collector is still repointed at MLFLOW_URL, which is the half that actually matters
ocp-service-bootstrap.sh	resolving to no MLflow credential at all is a precheck failure , not the warning it used to be. It previously wrote bearer_token: "" and let the install proceed
autobench-cli mlflow-health	the gate. One authenticated GET /mlflow/health against the deployed Service
preflight.py	calls that endpoint for both platforms; --skip-mlflow-probe opts out

The credential is supplied, not discovered

A pre-installed MLflow is authenticated by whoever installed it, so its credential cannot be read off the cluster the way the experiment id and workspace can (§2.3). `ocp-service-bootstrap.sh` therefore exposes all four shapes the Service itself resolves, in this precedence — pick **one**:

flag(s)	instance config	for
--mlflow-bearer-file F	bearer_token	a pre-obtained bearer, read in-process from a 600/400 file
--mlflow-token-secret S (the default, mlflow-reader-token)	bearer_token	a ServiceAccount-token Secret in rossoctl-system
--mlflow-username U --mlflow-password-file F [--mlflow-oauth-url URL]	username + password	the OpenShift OAuth challenge flow an RHOAI oauth-proxy expects
--mlflow-client-id ID --mlflow-client-secret-file F --mlflow-token-url URL	client_id + client_secret	an MLflow fronted by Keycloak
--mlflow-no-auth	the ignored bearer_token placeholder of §4	declaring the MLflow unauthenticated — what KinD's mlflow-reader is, and the only way to install without a real credential

Values come from files or the cluster, **never argv**, and are displayed only as an 8-char SHA-256 prefix, the same discipline as §3.6. Half a shape is an error rather than a fallback: `--mlflow-username` without `--mlflow-password-file` stops the script, because quietly reverting to the token Secret would install a credential nobody chose. `kind-service-bootstrap.sh` takes `--mlflow-no-auth` with the same spelling; without it, it still *infers* no-auth from an `mlflow-reader` tracking URL, and now says so in the log.

Why the check is a round trip

`GET /mlflow/health` is authenticated as `benchmarker`, runs **four independent stages**, and always answers 200 — the verdict is in the body, because "the probe could not run" and "MLflow is broken" are different findings:

stage	what it proves
auth	a bearer could be minted at all, through <code>auth/mlflow.py</code> — the Service's own resolution, not a re-implementation. <code>credential_mode</code> names which shape was selected: <code>bearer_token</code> , <code>openshift_oauth</code> , <code>client_credentials</code> or none. A declared no-auth MLflow reads as <code>bearer_token</code> , because that is what the placeholder selects; none means nothing was configured, and it fails
read	the traces API answers. This is the half a token report reads
write	the OTLP exporter posted one synthetic trace without logging an error
round_trip	that trace came back out of the read side again

The last two are the reason this exists. The two failures that shipped zero-byte reports on 2026-09-30 (§3.4) were both on the **write** half, and the read path answered 200 throughout — reads go through `httpx` honouring `insecure_tls`, while the span exporter is a different library needing a real trust anchor and an RBAC grant. A read-only check would have passed on both clusters. Worse, the OTLP exporter never raises: `BatchSpanProcessor` runs it on a worker thread and logs the failure, which is why the endpoint captures the `opentelemetry` logger for the probe's duration and returns the cause to you instead of leaving it in a pod log. A missing RBAC grant reports `Failed to export span batch code: 403, reason: Forbidden`; a missing TLS anchor reports the exporter's terminal line with the TLS cause spliced in, naming `CERTIFICATE_VERIFY_FAILED`.

The two read differently because a 403 is not retried and a TLS failure is: the exporter logs a retryable cause at `WARNING`, once per attempt, and leaves its terminal `ERROR` generic. The endpoint therefore watches both levels — without that, the ykt5 failure would come back saying only `Failed to export span batch due to timeout, max retries or shutdown.` and naming no cause at all.

```
BM_BASE=https://autobench-rossoctl-system.apps.example.com \
BM_PASSWORD_FILE=~/.rossoctl-ykt5/benchmarkmarker.pass \
uv run autobench-cli mlflow-health          # exit 8 = not healthy; --no-round-trip reads only
```

The probe **cannot pollute a report**, which is what makes it safe to run against a live install: every report is filtered to its own run's session ids, and the probe's is a fresh uuid. It also costs no LLM gateway call. It does write one trace per call, so `?round_trip=false` exists for a cheap read-only check.

4. The instance-config Secret

One JSON file per issuer, keyed by `iss`, mounted read-only at `/etc/service/instances`. **No install path creates it:** it carries the ROPC service credential and the S3 keys, so it is generated out-of-band and must never pass through Helm values or a values file.

One script per platform, so the file has reproducible provenance instead of being hand-assembled:

```
# OpenShift – prechecks the whole chain (ten rows on a single-namespace cluster) before writing
reference/ocp-service-bootstrap.sh --cluster ykt5 --context <ctx> \
  --apps-domain apps.ykt5.example.com \
  --password-file ~/.rossoctl-ykt5/benchmarkmarker.pass \
  --llm-api-base https://<external-gateway>/v1 \
  --out-dir instances/
# KinD – verifies the credential before writing it into the file
reference/kind-service-bootstrap.sh --context kind-rossoctl \
```

```
--password-file ~/.rossoctl-kind/benchmarkers.pass \
--copy-from instances/keycloak.localtest.me_8080.json \
--out-dir instances/
```

Both also accept `--password-stdin`, `--password`, or `KC_SERVICE_PASSWORD` in the environment, and default the user to `benchmarkers` (`--username` overrides) — see §3.6.

Then, on either platform:

```
kubectl -n rossoctl-system create secret generic autobench-instances \
  --from-file="<encoded-iss-host>.json=instances/<encoded-iss-host>.json" \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl -n rossoctl-system rollout restart deploy/autobench-service
```

The filename encodes the `iss` **host** with `:` rewritten to `_`; the exact `iss` inside the file is the source of truth. The Service reads instance files at startup, hence the restart. Note that `PUT /config` is in-memory only — a correction that must survive a restart goes in the Secret.

Four properties of these scripts are load-bearing:

- **Values reach `jq` through the environment, not `--arg`**. `argv` is world-readable (`ps`, `/proc/<pid>/cmdline`); a process's environment is not.
- **The file is created `600` by `umask`**, not `chmod` ed afterwards, so there is no window in which credentials sit world-readable on disk.
- **`--copy-from` / `--s3-from` carry `s3` and `workload_llvm` over** from an existing file. Those two are environment facts a script cannot discover, and a cluster rebuild should reproduce them rather than lose them — losing them is silent, and costs a run's artifacts.
- **A no-auth MLflow still needs a `bearer_token`**, which is not a contradiction: the Service mints a token *before* it reads, and with neither a bearer nor client-credentials the token helper raises, the route catches it, and the run fails soft into an empty token report. Both bootstrap scripts therefore emit the same ignored placeholder, `unused-no-auth-reader`, when the MLflow is declared unauthenticated (§3.7) — it is what KinD's reader gets. Writing *no* credential key is the one thing that does not work, and `GET /mlflow/health` reports it as a failed `auth` stage rather than leaving you to infer it from an empty report.

5. Installing with Helm

```
deploy/helm/autobench/
Chart.yaml           version = chart version; appVersion = the default image tag
values.yaml          the two platform shapes, documented inline
templates/           deployment, service, route, httproute, ibac-judge, ibac-judge-hooks,
                     _helpers.tpl, NOTES.txt
files/               proxy.py (the judge), platform-config-ibac.py (its two lifecycle hooks)
deploy/helm/values-openshift.yaml the OpenShift shape's overrides
deploy/helm/values-kind.yaml      KinD's overrides
```

`platform` selects the shape and is the only value most installs need to think about. It drives the pod security context and which ingress object is rendered (§3.4). The chart **references but never creates** `autobench-instances`.

Parity with the raw manifests is the chart's correctness gate, and it has teeth. Run this after any template change:

```
helm lint deploy/helm/autobench
python3 reference/helm-parity-check.py      # 12 checks; "Chart and manifests agree."
```

It renders both platform shapes and diffs them against

`deploy/{deployment,service,kind/httproute}.yaml` and against the OpenShift patched render. The standard `app.kubernetes.io/*` labels are deliberately **absent**: the sole label and selector is `app: autobench-service`, because that is what the live Deployments select on, and a Deployment's selector is immutable. If the check fails, the chart is wrong — not the manifests.

5.1 OpenShift

```
python3 reference/preflight.py --platform openshift --context <ctx> \
    --password-file ~/.rossoctl-ykt5/benchmarkmarker.pass      # 0 failures first
helm upgrade --install autobench deploy/helm/autobench \
    -n rossoctl-system --kube-context <ctx> -f deploy/helm/values-openshift.yaml
oc -n rossoctl-system rollout status deploy/autobench-service
HOST=$(oc -n rossoctl-system get route autobench -o jsonpath='{.spec.host}')
curl -fsS "https://$HOST/healthz"                             # {"status":"ok"}
```

Leave `route.host` empty unless you need a specific name — the router generates `autobench-
<namespace>.apps.<cluster>`, which is the shape every recipe in the developer guide assumes.

5.2 KinD

DEV/TEST only. On a **freshly rebuilt** cluster do not start here: `reference/kind-post-setup.sh` builds and `kind load` s the image, re-seeds the `benchmarkmarker` user, ensures the team secrets, generates the instance config and creates the Secret. The chart is the last step of that, and alone it installs a pod with nothing to authenticate as.

```
IMAGE=ghcr.io/rossoctl/autobench:v1.30 reference/kind-post-setup.sh  # first time / after a
rebuild
python3 reference/preflight.py --platform kind --context kind-rossoctl \
    --password-file ~/.rossoctl-kind/benchmarkmarker.pass
helm upgrade --install autobench deploy/helm/autobench \
    -n rossoctl-system --kube-context kind-rossoctl -f deploy/helm/values-kind.yaml
curl -fsS http://autobench.localtest.me:8080/healthz             # {"status":"ok"}
```

8080 there is the **host** port KinD publishes the istio gateway on; it is not this Service's port, and the two matching is a coincidence. Note also that `kind load docker-image` bypasses the registry, so a KinD pod's digest will not match the published one even at the same tag — compare source, not digests, there.

5.3 Adopting an install made from the raw manifests

A chart cannot replace an existing Deployment — the selector is immutable — but it can take ownership of one. Both clusters that predate the chart were adopted this way rather than reinstalled, which is why neither had an outage: **the running pod is not recreated.**

```
for obj in deploy/autobench-service svc/autobench-service httproute/autobench; do # or
route/autobench
```

```
kubectl -n rossoctl-system annotate "$obj" \
  meta.helm.sh/release-name=autobench meta.helm.sh/release-namespace=rossoctl-system --
  overwrite
kubectl -n rossoctl-system label "$obj" app.kubernetes.io/managed-by=Helm --overwrite
done
helm upgrade --install autobench deploy/helm/autobench -n rossoctl-system -f <values>
```

Without the annotations Helm refuses the install with "invalid ownership metadata". Afterwards, confirm the pod's `restartCount` and `creationTimestamp` are unchanged — adoption should be a metadata-only operation.

Read release ownership from the Secrets, not from `helm list`. A `helm list --kube-context` comparison across two clusters has returned identical output for different clusters here; the release objects cannot lie about where they live:

```
kubectl -n rossoctl-system get secret -l owner=helm,name=autobench --context <ctx>
# sh.helm.release.v1.autobench.v1
```

5.4 The IBAC judge — installed and uninstalled with the release

Needed **only** by the plugin legs of the matrix (#5–#8). Skip it entirely if you are not running those; nothing else in AutoBench calls it.

The judge is AutoBench's, not the platform's. Rossoctl ships none — `rossoctl-platform-config` arrives with every `ibac.*` field empty — and a cluster with no benchmark workloads never needs one. So the chart installs it and `helm uninstall` removes it, which is the whole point: no leftover Deployment for the next person to find and wonder about.

```
# The upstream key first. It is a credential, so the chart references it and never creates it –
  and
# for the same reason does not delete it. It must be a key for the SAME LLM service the
  workloads
# already call (§2): the judge is one more caller of it, and no two services share a key table.
umask 077; kubectl -n rossoctl-system create secret generic ibac-judge-upstream \
  --from-file=apikey=<that profile's key file>
helm upgrade --install autobench deploy/helm/autobench \
  -n rossoctl-system --kube-context <ctx> -f deploy/helm/values-openshift.yaml \
  --set ibacJudge.enabled=true \
  --set-string ibacJudge.upstreamBase="$INTERNET_LLM_API_BASE" \
  --set-string ibacJudge.model="$INTERNET_LLM_MODEL" --wait
```

`upstreamBase` and `model` are required and appear in no committed values file, because one is a live host and this repo is public. Pass them from the selected profile (§2). `model` must be an id **that gateway lists in `GET /v1/models`** — a model the judge's key cannot see fails one judge call at a time, mid-leg, and the plugin falls back to admitting the call.

What is chart-owned and what is not. The judge Deployment, Service and code ConfigMap are ordinary release objects. The two `ibac.*` fields are not objects at all: they are keys inside `rossoctl-platform-config`, which is templated by the **rossoctl release**. Helm owns whole objects, never fields inside another release's, so the chart handles those two keys with a pair of hooks — `post-install` / `post-upgrade` records what they said in `configmap/ibac-judge-prior` and patches them; `pre-delete` writes the recorded pair

back. Do not delete that record by hand: without it the uninstall leaves the fields alone rather than guess, and they then name a judge that no longer exists.

object	created by	removed by <code>helm uninstall</code>
deploy/ibac-judge, svc/ibac-judge, cm/ibac-judge-code	the chart	yes
sa/role/rolebinding ibac-judge-config	the chart	yes
secret/ibac-judge-upstream	you, out-of-band	no — delete it yourself when done
ibac.judgeEndpoint / judgeModel in rossoctl-platform-config	post-install hook	restored to their previous values by the pre-delete hook

Two failure modes to know, both silent:

- A **helm upgrade of the rossoctl release reverts the patch**. That ConfigMap is re-rendered from the platform chart's own values, so the judge fields go back to empty. The plugin then loads and does nothing: every tool call admitted, legs #5–#8 still pass, and the only tell is a judge call count of zero. Re-run `preflight.py --plugin-legs` after any platform upgrade — that is why the check reads the live cluster rather than trusting install-time state.
- A **failed pre-delete hook aborts the uninstall**. Deliberate: the alternative is deleting the judge while the platform still points at it. Read the Job's log, fix it, retry — or `helm uninstall --no-hooks` and put the fields back by hand.

Verify both directions:

```
kubectl -n rossoctl-system get cm rossoctl-platform-config \
-o jsonpath='{.data.config\.yaml}' | grep -A6 '^ibac:'      # judgeEndpoint/judgeModel now set
kubectl -n rossoctl-system get cm ibac-judge-prior -o jsonpath='{.data}' # the recorded prior
pair
kubectl -n rossoctl-system logs job/ibac-judge-config-apply # and how it got there
```

The apply Job is **not** deleted on success — its log is the only account of what the fields said at the moment of the patch, and the next upgrade replaces it. The restore Job is the opposite: it deletes itself on success, because after an uninstall nothing else ever would.

Enforcement itself is proven by **counting judge calls**, never by a pass rate — an inert plugin and a working one produce the same pass rate. See `docs/PLUGIN_OVERHEAD.md`.

5.5 Upgrade, rollback, uninstall

```
helm upgrade autobench deploy/helm/autobench -n rossoctl-system -f <values> \
--set image.tag=v1.30
helm history autobench -n rossoctl-system
helm rollback autobench 1 -n rossoctl-system
helm uninstall autobench -n rossoctl-system # leaves autobench-instances behind
```

`helm uninstall` deletes only what the chart owns, so the instance Secret survives — which is what you want, since regenerating it means re-reading credentials. The image tag is immutable, so `imagePullPolicy: IfNotPresent` is correct and an upgrade means changing `image.tag`, never restarting to pick up a rebuild.

With `ibacJudge.enabled` the uninstall does more: it runs the `pre-delete` hook that puts `rossoctl-platform-config`'s `ibac.judgeEndpoint / judgeModel` back to what they were before the install, then deletes the judge along with everything else (§5.4). Two consequences. A failed hook **aborts** the uninstall rather than leaving the platform pointed at a judge that is about to go — read `logs job/ibac-judge-config-restore`, or use `--no-hooks` and fix the fields by hand. And `secret/ibac-judge-upstream` survives, like the instance `Secret` and for the same reason: it is a credential the chart never created.

`helm rollback` does **not** re-run the pre-delete hook, and a rollback across an `ibacJudge.enabled` boundary is the one case worth avoiding — roll forward with an explicit `helm upgrade` instead, so the apply hook reconciles the fields.

6. Verifying the install

`/healthz` proves the pod runs. It proves nothing about the chain the pod exists to drive, and every link in that chain has a failure mode that still returns 200. Work outward:

```
TOKEN=... # ROPC; see DEVELOPER_GUIDE.md §3
curl -fsS "$BASE/healthz"
curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/benchmarks" | jq '.items[].name'
curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/benchmarks/gsm8k" | jq
  '.agents[0].container_image'
```

`GET /benchmarks` returns `items`, not `benchmarks` — that has broken scripts twice — and it only summarises each benchmark (`name`, `mcp_image`, `agents`, `default_model`). An agent's `container_image`, `extra_env` or `model_override` is visible only through `GET /benchmarks/<name>`, so confirming a catalog change against the list endpoint alone can pass while the thing you changed is still the old value.

Then the only check that means anything: **a 1-task gsm8k leg whose report carries a non-zero token row.**

```
uv run autobench-cli --base "$BASE" run gsm8k --max-tasks 1 # see DEVELOPER_GUIDE.md §7.1
```

what you look at	expected	what it proves
run status	completed, pass rate 1.0	Service → Rossoctl → operator → agent → MCP → evaluator
tokens on the task row	non-zero (~320 in / 87 out)	agent → collector → MLflow → Service, the whole telemetry chain
model on the row	the gateway's model id, not "unknown"	the Service read the experiment the collector wrote to
pod imageID digest	matches the tag you installed (OpenShift)	the pod is not serving a stale : latest

A zero-token row is the single most informative failure in this system, because it is the shared signature of four unrelated causes: a refused MLflow read, an experiment-id mismatch, a collector that cannot be reached, and `workload_agent_runner` set to the value the current agent image does not emit spans for. Preflight separates the first three; only a run separates the fourth.

7. Symptoms that lie

symptom	actual cause	how to confirm
run passes, every token count 0, model: "unknown"	refused MLflow read, wrong experiment id, unreachable collector, or the wrong workload_agent_runner	§2.4, then §6
run passes pass_rate 1.0, and report.ndjson/token_report.ndjson are zero bytes — only 4 of the 8 artifacts carry anything	the Service's <i>own</i> span export failed, so MLflow never got the root Agent.Session span and the trace was dropped. On OpenShift, one of the two halves in §3.4: no service-CA trust anchor (TLS) or no trace-writer RoleBinding (403)	autobench-cli mlflow-health (§3.7) — its write stage names the cause where the run named nothing, and it is the only check that reproduces this without a run. preflight.py reports both halves separately as well
the run publishes no artifacts at all , boto core ... SSL error: unable to get local issuer certificate	REQUESTS_CA_BUNDLE was used for the service CA instead of OTEL_EXPORTER_OTLP_TRACES_CERTIFICATE; boto core honours it too and the S3 client can no longer validate AWS's cert	§3.4
one task errors A2A task ended in state 'failed': Error: timed out at ~30 s, with llm_count: 0	the LLM gateway accepted the connection and never answered. The agent's service runner caps a single react at a hard-coded 30 s with no retry (docker and venv allow 600 s), so a stalled completion becomes a failed task	probe the gateway from inside the agent pod: a stall is a <i>read</i> timeout after TLS succeeds, and it also hits the unauthenticated GET /public/litellm_model_cost_map, which proves it is not the model
/deploy returns 502	Keycloak or the operator returned 403 — the service credential no longer logs in, or the user lacks the rossoctl-operator realm role. The install itself looks healthy: the password is only used per deploy	preflight.py --password-file ... (§3.6); then the role mapping in the realm
/deploy returns 424 , agent CrashLoopBackOff	collector endpoint on 4318, or hf-secret missing so the MCP pod never started	the collector's command; get secret hf-secret
token request 400 Account is not fully set up	the realm requires firstName/lastName for ROPC	the user's profile
tasks error on the first completion, run finishes with zeroes	openai-secret empty, or holding the <i>other</i> gateway's key	preflight.py prints both namespaces' sha8; compare them
Model endpoint ... is unreachable on every task	the agent pod is running dev145 , whatever :latest points at	the wording: dev146 says did not respond ... after N attempt(s). Compare the pod's imageID digest, never the tag
a config change appears to do nothing	InstanceConfig and the SERVICE_* settings both use extra="ignore" — an unknown field is dropped silently	read the value back; adding one is never self-verifying
the new api_base is ignored	a stale Deployment from an earlier bring-up is still serving the old one	list Deployments in the team namespaces by age
a pod serves old code at the right tag	:latest agent/MCP images: a <i>running</i> pod never re-pulls	tear down before deploying; compare .status.containerStatuses[].imageID

symptom	actual cause	how to confirm
mlflow-reader restarts ~30 s after boot with no error in the log	OOMKilled — one MLflow 3.x worker idles at ~2.3 GiB, so a 2Gi limit is below the floor	the container's lastState, not its log
get crd routes.route.openshift .io says Routes are missing	Routes come from the aggregated apiserver and have no CRD	kubectl api-resources --api- group=route.openshift.io
/healthz 503, deploy 500 httpx.ReadError, agent CrashLoop — all at once, on KinD	a ztunnel restart un-enrolled older pods from the ambient mesh	recycle every pod older than ztunnel's Ready condition, in all namespaces

8. Appendix: the raw-manifest path, and two KinD-only objects

The manifests under `deploy/` remain supported and are what the chart is checked against:

```
kubectl apply -f deploy/service.yaml -f deploy/deployment.yaml
oc patch deploy autobench-service --patch-file deploy/openshift/deployment-patch.yaml #
OpenShift
kubectl apply -f deploy/kind/httproute.yaml # KinD
```

On OpenShift the patch must go on **before** the rollout, not after: it is what removes the pinned UID that restricted-v2 would otherwise reject.

Two objects live under `deploy/kind/` and are in neither the chart nor the OpenShift path, because neither has an OpenShift counterpart.

`mlflow-reader.yaml` — stock MLflow, no auth, serving the *same* postgres as the writer. It exists because the MLflow `rossocctl-deps` installs runs `mlflow-oidc-auth`, which rejects both the Service's read and the collector's write; the run then publishes zeros on every task, which reads like missing telemetry rather than a refused read. It is a "reader" by convention only. Two of its settings are measured rather than chosen: `--workers 1` (MLflow 3.x defaults to four) and a **4Gi** limit. Gate on a 200 from `/api/2.0/mlflow/traces` rather than on the pod going Ready — two pip installs run at container start — and pass `experiment_ids`, since that endpoint answers 400 without it. Probe from **inside** the pod (MLflow 3.x rejects the API server's service proxy as a DNS-rebinding attempt) with `python`, not `curl`, which the image does not ship. `reference/kind-post-setup.sh` does all of this, including pointing the collector at the reader; by hand it is:

```
python3 reference/kind-collector-mlflow.py # patch + restart; idempotent
python3 reference/kind-collector-mlflow.py --check # report only, exit 1 if unpatched
```

`ibac-judge.yaml` — the upstream proxy the AuthBridge `ibac` plugin calls. The plugin's ConfigMap is cleartext, so the key cannot live there; this holds it in a Secret and adds the `Authorization` header outbound, leaving the plugin pointed at a credential-free in-cluster URL. Without it the plugin is **inert**: it admits every call and the run looks like a clean pass, which is why enforcement is proven by *counting judge calls* and never by reading a pass rate. `UPSTREAM_BASE` in the committed copy is an `example.com` placeholder — substitute the real gateway at apply time, and `rollout restart` after the key lands, since the proxy reads it once at pod start.